

Distributed Flexible Job Shop Scheduling With Heterogeneous Transportation Resources Constraints via Deep Reinforcement Learning and Graph Neural Network

Kaikai Zhu, Xiaobin Li^{ID}, *Member, IEEE*, Pei Jiang^{ID}, *Member, IEEE*, Min Cheng^{ID}, Yuanqing Wu^{ID}, *Senior Member, IEEE*, Kaizhou Gao^{ID}, *Member, IEEE*, and Lei Ren^{ID}, *Member, IEEE*

Abstract—The distributed flexible job shop scheduling problem (DFJSP) has emerged as a critical challenge in the field of scheduling optimization due to its intricate resource allocation and the demand for production–logistics collaboration across multiple factories. However, most existing studies related to DFJSP only focus on the production and transportation process of jobs within a single factory, while neglecting the cross-factory logistics and the heterogeneous characteristics of transportation resources. Therefore, this article first investigates the distributed flexible job shop scheduling problem with heterogeneous transportation (DFJSPHT) resource constraints and proposes an end-to-end deep reinforcement learning (DRL) scheduling method to minimize the makespan. An innovative heterogeneous disjunctive graph model is constructed to uniformly represent the states of factories, machines, operations, and transportation resources in DFJSPHT, and the scheduling process is modeled as a Markov decision process (MDP). Next, a resource release strategy is developed to enhance the efficiency of transportation resources. To enhance the feature expression ability of the model, a graph neural network (GNN) is employed to capture the problem characteristics, and the policy network is trained using

the proximal policy optimization. Comparative experiments are conducted on synthetic and benchmark instances demonstrate that the proposed method outperforms the classical priority scheduling rules and two popular DRL-based scheduling methods in solving DFJSPHT, with performance improvements exceeding 10% in most instances.

Index Terms—Deep reinforcement learning (DRL), distributed flexible job shop, graph neural network (GNN), heterogeneous disjunctive graph, heterogeneous transportation resource constraints.

I. INTRODUCTION

WITH the rapid development of intelligent manufacturing and Industry 5.0 technologies, the distributed collaborative production mode is gradually becoming an important development direction for the advanced manufacturing industry due to its significant advantages in meeting personalized customer needs, responding to complex process flows, and achieving efficient resource collaboration among multiple factories. The distributed flexible job shop scheduling problem (DFJSP), as a typical distributed production optimization problem widely existing in discrete manufacturing industries such as electronic products and automobiles, has become a research hotspot in the field of production scheduling owing to its high complexity and practical application value [1], [2], [3], [4]. As an important extension of the flexible job shop scheduling problem (FJSP), DFJSP involves three subproblems: factory selection, machine allocation, and operation sequencing, which are more challenging in modeling and solving than FJSP. In a distributed manufacturing environment, transportation is a critical link in ensuring the orderly operation of production. The efficient transportation of raw materials, semi-finished products, and finished products through transportation resources, such as automated guided vehicles (AGVs), forklifts, and trucks, can enhance overall production efficiency and resource utilization [5], [6]. Existing research on DFJSP considering transportation resource constraints mostly focuses on transportation within a single factory and generally relies on metaheuristic algorithms for solution [7]. However, in the actual distributed production model, cross-factory transportation demands are common due to differences in product process paths, and the types of transportation resources

Received 13 November 2025; accepted 13 January 2026. Date of publication 29 January 2026; date of current version 16 April 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3308001, in part by the National Natural Science Foundation of China under Grant 52075060 and Grant 52575561, and in part by the Special Project for Technological Innovation and Application Development of Chongqing Municipality under Grant CSTB2024TIAD-STX0029. This article was recommended by Associate Editor Y. J. Chen. (*Corresponding author: Xiaobin Li.*)

Kaikai Zhu, Xiaobin Li, Pei Jiang, and Min Cheng are with the State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, Chongqing 400030, China (e-mail: kaikaizhu@stu.cqu.edu.cn; cqlixiaobin@163.com; peijiang@cqu.edu.cn; chengmin@cqu.edu.cn).

Yuanqing Wu is with the School of Intelligent Systems Engineering, Sun Yat-sen University, Guangzhou 510275, China (e-mail: yqwuzju@163.com).

Kaizhou Gao is with the Department of Engineering Science, Macau University of Science and Technology, Macau, SAR, China, and also with Macau University of Science and Technology and Zhuhai MUST Science and Technology Research Institute, Zhuhai 519099, China (e-mail: kzgao@mst.edu.mo).

Lei Ren is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China, also with the Zhongguancun Laboratory, Beijing 100194, China, and also with the State Key Laboratory of Intelligent Manufacturing System Technology, Beijing 100854, China (e-mail: renlei@buaa.edu.cn).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TSMC.2026.3656196>.

Digital Object Identifier 10.1109/TSMC.2026.3656196

2168-2216 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on April 22, 2026 at 02:15:05 UTC from IEEE Xplore. Restrictions apply.

required are usually different from internal transportation within the factory. In addition, when facing complex multi-factory scenarios and real-time scheduling demands, classical metaheuristic algorithms generally suffer from slow response times and poor generalization capabilities.

To address the aforementioned challenges, this article proposes an end-to-end intelligent scheduling method based on graph neural network (GNN) and deep reinforcement learning (DRL) to solve the DFJSP with heterogeneous transportation resource constraints (DFJSPHT), with the objective of minimizing the makespan. The contributions of this article are summarized as follows.

- 1) The DFJSPHT model is proposed for the first time, where the scheduling process is modeled as a Markov decision process (MDP) through the designed state representation, actions, and rewards.
- 2) A heterogeneous disjunction graph model, including operation nodes, machine nodes, and transportation resource nodes, is constructed to represent the state characteristics and interaction relationships among the three types of nodes, thereby enhancing the feature expression ability of the model.
- 3) A three-stage node feature embedding method is designed, which can effectively capture the correlation among nodes through an iterative feature extraction process, fully addressing the complex constraints and dependencies of multiple factories, resources, and operations in DFJSPHT.
- 4) A release strategy based on resource states is proposed, which reasonably releases occupied resources to reduce the waiting time of transportation resources, thereby improving the overall efficiency and resource utilization of the production process.

The rest of the article is organized as follows. Section II reviews the related work with DFJSP and FJSP. Next, the problem description and the disjunction graph model of DFJSPHT are provided in Section III. In Section IV, the proposed solution methods based on GNN and DRL are introduced. The experiments conduct and the analysis of results are listed in Section V. Section VI provides a summary of the article.

II. RELATED WORKS

A. DFJSP and DFJSP With Transportation Constraints

With the continuous development of the distributed manufacturing mode and the growing demand for cross-factory collaboration, DFJSP has gradually become a key research hotspot in the field of scheduling [8], [9], [10]. Sang and Tan [11] proposed a high-dimensional multiobjective memetic algorithm to solve the constructed multiobjective distributed flexible job shop collaborative scheduling model, and the experimental results verified the feasibility of the proposed model and the superiority of the algorithm. Xie et al. [12] designed a hybrid genetic tabu search algorithm combining genetic algorithm and tabu search to handle the DFJSP, in which a new neighborhood structure and two genetic operators were introduced to expand the solution set space. Yu et al. [13] studied the energy-efficient scheduling problem of distributed flexible job shop and proposed a knowledge-guided

bipopulation evolutionary algorithm to solve it with the goal of minimizing the makespan and total energy consumption. Considering the randomness of processing time in DFJSP, Fu et al. [14] developed an evolutionary algorithm combined with an evaluation method to solve this problem in order to achieve the minimum completion time and total delay.

Some researchers have taken into account the importance of transportation resources in the production process and extended the traditional DFJSP. Marzouki et al. [15] proposed an improved chemical reaction optimization metaheuristic method to solve the DFJSP with multiple transport robots and constructed two types of instances to verify its performance. Luo et al. [16] designed an efficient memetic algorithm to solve the DFJSP with transfers, where different operations of the job can be processed in different factories. Yang and Xu [17] proposed a hybrid decomposition variable neighborhood memetic algorithm to solve the DFJSP, considering fuzzy processing time and transportation time, aiming to minimize the makespan, maximize factory workload and total workload. Pan et al. [7] developed a novel bilearning evolutionary algorithm to solve distributed energy-efficient FJSP with transportation constraints, which integrated statistical and evolutionary learning methods and can employ both decomposition and Pareto dominance methods to deal with the conflicting objectives.

In summary, although there have been studies focusing on DFJSP with transportation, most of them only involve the transportation process between machines within a single factory, ignoring the actual needs of cross-factory transportation and the heterogeneous characteristics of limited transportation resources. In addition, most existing studies related to DFJSP employ metaheuristic algorithms for solving, but the real-time requirement of multifactory production-logistics collaboration is difficult to meet. Therefore, there is an urgent need for an efficient and accurate method to handle DFJSP with heterogeneous transportation resource constraints, thereby enhancing its practicality and robustness in complex production environments.

B. DRL Methods for Solving the FJSP and DFJSP

Most of the existing methods for solving FJSP and DFJSP are traditional heuristic algorithms incorporating local search operators, which can obtain high-quality solutions after many iterations. However, the heuristic algorithms generally suffer from poor adaptability and low response efficiency in solving such a complex multiconstraint shop scheduling problem. To overcome the above limitations, DRL methods have gradually become an emerging solution approach for FJSP and DFJSP problems due to their excellent generalization ability and real-time decision-making efficiency [18], [19], [20].

In terms of solving FJSP, Yuan et al. [21] proposed a novel DRL framework to handle FJSP, which used a lightweight multilayer perceptron (MLP) as a state embedding network to extract state information and define a new state representation to directly reflect the state features of candidate actions. Luo et al. [22] developed a real-time scheduling method with hierarchical multiagent proximal policy optimization, including the objective agent, job agent, and machine agent. In addition, a new method combining the GNN and DRL

is gradually being used to solve FJSP. Song et al. [23] proposed a heterogeneous graph neural network architecture (HGNN) to capture the relationship between machines and operations in FJSP, and utilized an end-to-end DRL method to solve it. Wan et al. [24] decomposed the tasks of FJSP into operation selection and machine assignment tasks, and integrated the metapath-based HGNN and DRL to solve the dual-task FJSP effectively. Wang et al. [25] proposed an effective feature extraction network architecture based on a graph attention mechanism and an MLP, and integrated a size-agnostic DRL-based approach to improve solution quality. Du et al. [26] utilized a deep Q -network (DQN) to solve FJSP with crane transportation and setup time, and used a weighted method to simultaneously optimize the makespan and total energy consumption. Wang et al. [27] designed a new hierarchical multiagent DRL framework to solve dynamic FJSP with transportation, and enhanced the solution performance of the framework through an imitation learning strategy. Yuan et al. [28] proposed an improved dual-depth DQN real-time scheduling method to solve FJSPs with AGVs. To address the collaborative scheduling problem of production–logistics in flexible job shops considering the interference of logistics equipment failures, Shi et al. [29] proposed a nested hierarchical DRL method, which designed a nested hierarchical framework containing three agents to achieve multiobjective optimization. Li et al. [30] developed a multiagent scheduling architecture to solve dynamic FJSP with AGV, in which an efficient action decoding method was proposed to improve the learning efficiency of the algorithm.

In terms of solving DFJSP, Li et al. [31] proposed a coevolution algorithm based on DQN to solve the energy-aware distributed heterogeneous FJSP, which included a new coevolutionary framework and nine problem feature-based local search operators to accelerate the convergence speed of the algorithm. Yan et al. [32] developed a DRL-based multiagent approach incorporating distribute and sequence agents for solving the DFJSP with random job arrivals, and designed a DQN framework combining linearly decreasing threshold probabilities to efficiently balance exploration and exploitation in the training phase. A hybrid HGNN was designed by Li et al. [33] for solving DFJSP with family setup time, in which a transform mechanism based on principal component analysis was developed to further integrate three types of embeddings and combined with a local search method to enhance the search capability of the algorithm. Li and Chai [34] studied energy-saving DFJSP with fuzzy processing time in the Industrial Internet of Things and proposed a multitask multiobjective algorithm with feedback mechanism based on reinforcement learning (RL) to solve it, aiming to simultaneously optimize the makespan and energy consumption. Regarding the DFJSP with transportation resources constraints, Zhang et al. [35] designed a Q -learning-based hyperheuristic evolutionary algorithm to solve the DFJSP with crane transportation that aims to minimize the weighted sum of makespan and total energy consumption.

The above literatures show that researchers have developed a variety of DRL methods to solve FJSP and DFJSP in order to meet the requirements of real-world manufacturing processes for the real-time and generalization capabilities of

scheduling methods, and have achieved significant results. Meanwhile, GNN have been proven to capture the high-order correlations among nodes, enhancing the understanding ability of the problem model for complex problems. However, there is currently no suitable and efficient solution method for the DFJSPHT first proposed in this article. In view of this, this article combines the advantages of DRL and GNN, and introduces a resource release strategy to develop a novel end-to-end intelligent scheduling method for effectively addressing the DFJSPHT.

C. Research Gap Analysis

From the above related work, it can be seen that although research on DFJSP has made some progress, there are still research gaps in the distributed production–logistics collaborative scheduling based on DRL, mainly reflected in the following three aspects.

- 1) Oversimplified transportation modeling. The existing research on DFJSP, considering transportation, usually assumes sufficient transportation resources and constant transportation time between any two machines, ignoring the limitations of transportation resource quantity and the differences in actual transportation time.
- 2) Missing heterogeneity constraints for transportation resources. To our knowledge, there is currently no research exploring the heterogeneous characteristics of transportation resources in a distributed production environment, particularly in terms of transportation range and accessibility.
- 3) Absence of DRL-based cross-factory collaborative scheduling methods. At present, no literature has utilized the DRL-based method to solve the cross-factory production–logistics collaborative scheduling problem, which makes it difficult to meet the requirements of actual manufacturing systems for real-time scheduling and solution accuracy.

III. PROBLEM DESCRIPTION AND MODEL ESTABLISHMENT

A. Problem Formulation

DFJSPHT is an important extension of traditional DFJSP, which simultaneously considers the constraints of two types of heterogeneous transportation resources on the production process. For the sake of unified expression, the two types of heterogeneous transportation resources are collectively referred to as AGV, and are further categorized into intrafactory transportation AGV (IAGV) and cross-factory transportation AGV (CAGV) according to their transportation scope. The responsibilities of the two types of AGVs are as follows: IAGV is responsible for the transportation tasks between different machines within the factory, and CAGV is responsible for the transportation tasks among different factories. Specifically, if the job needs to continue processing on another machine within the factory, it needs to be transported by an IAGV; if it needs to continue processing on a machine in another factory, it needs to be transported by a CAGV. A practical example of DFJSPHT is depicted in Fig. 1, and the explanation of the notations involved in the

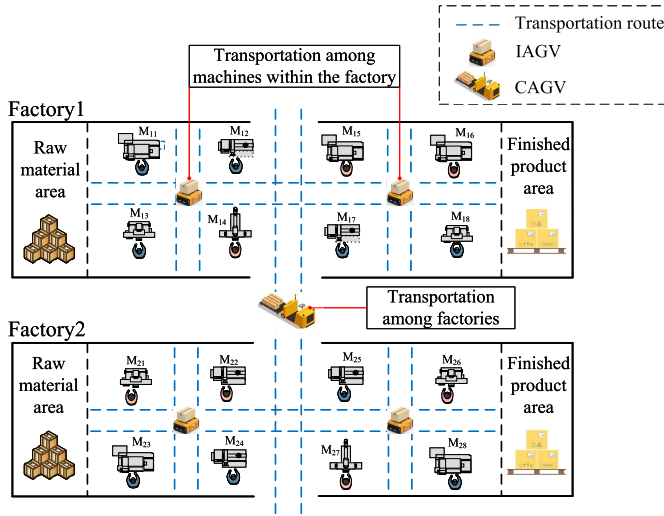


Fig. 1. Practical example of DFJSPHT.

problem description is shown in “Section I of Supplementary Materials.”

In a DFJSPHT instance with a size of $n \times m \times y \times r \times v$, there are a total of n jobs, consisting of n_i operations, which need to be transported by r IAGVs and v CAGVs to m machines in y different factories for processing. Specifically, each job needs to be transported by an IAGV from the raw material area to the designated machine for processing the first operation, and then transported by an IAGV to the finished product area after all operations of the job are completed. For the operation O_{ij} , it can be processed on any available machine $M_{fk} \in M_{ijf}$ in any available factory $F_f \in F_{ij}$ with a processing time P_{ijfk} , and can be transported to other machines for further processing by any IAGV or CAGV $A_{fc} \in A_{ijf}$ with a transportation time $T_{kk'}$. $T_{kk'}$ is determined by a predefined matrix $D \in R^{(g+2) \times (g+2)}$, which denotes the transportation time of the jobs between the two machines, and M_0 and M_{g+1} correspond to raw material and finished product areas, respectively. Therefore, DFJSPHT can be decomposed into four subproblems: 1) designate a factory for each operation; 2) designate a machine for each operation; 3) designate a transportation resource for each operation that needs to be transported; and 4) determine the processing sequence of all operations. Furthermore, the assumptions related to DFJSPHT are as follows.

- 1) An operation can only be transported or processed after its previous operation is completed.
- 2) Each IAGV can perform all transportation tasks within its affiliated factory and can only transport one job at a time.
- 3) Each CAGV can perform all cross-factory transportation tasks and can only transport one job at a time.
- 4) All IAGVs have the same transportation time between two machines within their affiliated factories.
- 5) All CAGVs have the same transportation time between two machines in different factories.
- 6) All machines, jobs, IAGVs, and CAGVs are idle at time 0.

TABLE I
PROBLEM INSTANCE OF DFJSPHT

Job	operation	F_1			F_2			CAGV
		M_{11}	M_{12}	IAGV	M_{21}	M_{22}	IAGV	
J_1	O_{11}	23	24		–	26		
	O_{12}	40	–		26	35		
	O_{13}	11	–		–	13		
	O_{21}	–	11		12	11		
J_2	O_{22}	21	–		24	–		
	O_{23}	14	–		10	12		
	O_{24}	39	32	A_{11}	–	31	A_{21}	A_{01}
	O_{31}	–	34		–	39		
J_3	O_{32}	38	–		32	–		
	O_{41}	37	–		33	34		
J_4	O_{42}	–	28		–	40		
	O_{43}	–	18		29	–		

TABLE II
TRANSPORTATION TIME AMONG MACHINES

	RA	M_{11}	M_{12}	M_{21}	M_{22}	FA
RA	0	11	17	13	18	0
M_{11}	11	0	12	38	35	12
M_{12}	17	12	0	36	36	12
M_{21}	14	38	36	0	20	19
M_{22}	18	35	36	20	0	19
FA	0	12	12	19	19	0

- 7) Transportation route conflicts and power replenishment are not taken into account.
- 8) The loading time, unloading time, and return time to the designated location of IAGVs and CAGVs are included in the predefined transportation time.

In order to further understand DFJSPHT, an instance is given as shown in Table I, which contains information about four jobs (J_1 – J_4) to be processed and two available factories (F_1 and F_2). F_1 and F_2 contain three machines and an IAGV, and a CAGV for cross-factory transportation. For example, the processing time of O_{11} on M_{11} in F_1 is 23, and the processing time of O_{41} on M_{22} in F_2 is 34. Table II shows the transportation time among machines, where RA and FA represent the raw material area and finished product area of all factories, respectively. For example, the transportation time from the raw material area to M_{11} is 11, and the transportation time from M_{12} to M_{22} is 36.

Fig. 2 is the Gantt chart of a feasible solution for the above instance, which clearly represents the processing and transportation process of all jobs in two factories. Taking job 2 as an example, it is first transported from the raw material area to M_{12} in F_1 through A_{11} for processing the first operation O_{21} . Then, it will be transported to M_{21} in F_2 through A_{01} for processing O_{22} and O_{23} . Subsequently, it is transported to M_{22} through A_{21} for processing O_{24} . Finally, the completed J_2 is transported to the finished product area in F_2 through A_{21} .

B. Heterogeneous Disjunctive Graph Model

In view of its ability to intuitively depict the sequence constraints among operations and the competitive relationship

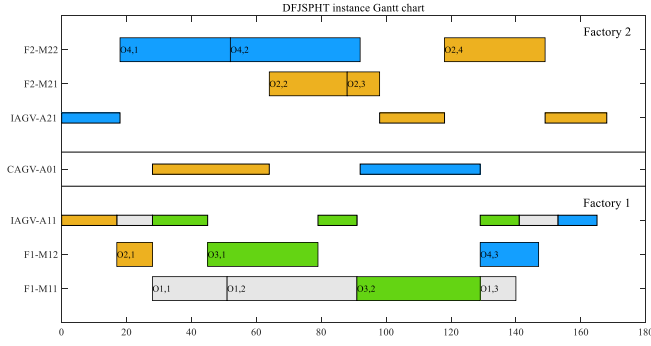


Fig. 2. Feasible solution for problem instance.

among shop resources, the disjunctive graph is widely used to represent the overall state in JSP and FJSP. Inspired by existing research [23], this article redefines a novel heterogeneous disjunctive graph $\mathcal{H} = (\mathcal{O}, \mathcal{M}, \mathcal{A}, \mathcal{C}, \mathcal{G})$, as shown in Fig. 3. The meanings represented by each module in the given example are as follows.

- 1) $\mathcal{O} = \{O_{11}, O_{12}, O_{13}, O_{1c}, O_{21}, O_{22}, O_{2c}, O_{31}, O_{32}, O_{3c}\}$ is a set of operation nodes. For each job, a virtual operation node O_{ic} is added to represent the process of transporting the completed job to the finished product area, and the operation O_{ic} is set to be processed only by the finished product area (i.e., end node).
- 2) $\mathcal{M} = \{M_{11}, M_{12}, M_{21}, M_{22}\}$ is a set of machine nodes. Machine nodes M_{fk} with different filling colors correspond to machines in different factories (e.g., M_{11} representing machine 1 in factory 1, and M_{22} representing machine 2 in factory 2). In addition, start and end are also set as machine nodes, representing the raw material area and finished product area within all factories, respectively.
- 3) $\mathcal{A} = \{A_{11}, A_{21}, A_{01}\}$ is a set of AGV nodes. AGV nodes with different filling colors correspond to IAGV in different factories or CAGV. In this instance, A_{11} and A_{21} represent IAGV1 of factory 1 and IAGV1 of factory 2, respectively. It is worth noting that the factory to which CAGV belongs is uniformly represented by 0, i.e., A_{01} represents the CAGV1 for transportation between factories 1 and 2.
- 4) $\mathcal{C}$ is a directed connecting arc set that represents the processing sequence between different operations of the same job, and the arcs connected with the start and end nodes is not included in $\mathcal{C}$.
- 5) $\mathcal{G}$ is a set of disjunctive arcs representing the relationship of operation-machine (O-M) and operation-AGV (O-A). The O-M arc signifies that the operation can be processed on the corresponding machine, and the O-A arc signifies that the job can be transported by the corresponding AGV to the designated machine for processing the corresponding operation.

Based on the above heterogeneous disjunctive graph model, the redundant O-M and O-A arcs need to be dynamically deleted during the processing and transportation scheduling of the jobs to reduce the action search space. Taking the scheduling process of job 3 in Fig. 3 as an example to elaborate

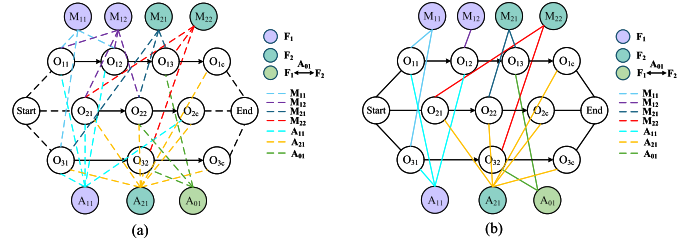


Fig. 3. Heterogeneous disjunctive graph of DFJSPHT. (a) Instance. (b) Solution.

on the details of action execution at each step. First, the $\{O_{31} - \text{Start}\}$, $\{O_{31} - A_{11}\}$, and $\{O_{31} - M_{11}\}$ arcs mean that job 3 is transported by A_{11} from the raw material area to M_{11} for processing O_{31} , while the $\{O_{31} - A_{21}\}$, $\{O_{31} - M_{21}\}$, and $\{O_{32} - A_{21}\}$ arcs are deleted. Subsequently, the $\{O_{32} - M_{22}\}$ and $\{O_{32} - A_{01}\}$ arcs mean that job 3 is transported by A_{01} from M_{11} in factory 1 to M_{22} in factory 2 for processing O_{32} . Finally, the $\{O_{3c} - \text{End}\}$ and $\{O_{3c} - A_{21}\}$ arcs mean that job 3 is completed and transported by A_{21} to the finished product area.

Compared with the disjunctive graph models considering transportation and processing constraints constructed in [23] and [25], the heterogeneous disjunctive graph modeling method proposed in this article can more accurately characterize the distributed flexible job shop scheduling process with heterogeneous transportation resource constraints, and can be further extended to DFJSP scenarios with other constraints. By embedding the factory information into the machine nodes and merging the heterogeneous transportation resource nodes, the complexity of the disjunctive graph structure and the size of the action space are significantly reduced, which improves the expressive capability and solving efficiency of the model.

IV. PROPOSED METHOD

To solve the proposed DFJSPHT, this section combines DRL and HGNN to obtain a reasonable and feasible end-to-end real-time scheduling scheme. First, the scheduling process of DFJSPHT is modeled as an MDP by defining the state, action, state transition, and reward. Subsequently, the designed resource release strategy that can effectively reduce the AGV waiting time is presented. Finally, the feature embedding process of machine node, AGV node, and operation node in the heterogeneous disjunctive graph and decision-making module are elaborated. The execution flow of the proposed method is shown in Fig. 4.

A. MDP Formulation

The scheduling process of DFJSPHT can be simply described as selecting a transportation resource for the operation that needs to be transported to the available machine for processing. At the decision point t , there exists at least one available operation-machine-AGV pair (O-M-A) that enables O_{ij} to be transported to M_{fk} in factory f via A_{fc} . Then, the probability distribution of the executable actions is obtained based on the environmental state information s_t and the current strategy π . Finally, the state s_{t+1} is updated by selecting a_t

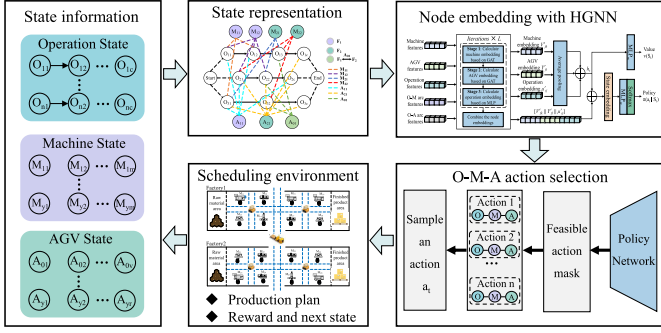


Fig. 4. Execution flow diagram of the proposed method.

and executing it with a greedy strategy, and the environment returns the reward r_t . The feasible solution of DFJSPHT can be obtained by iterating the above process, and its corresponding MDP is defined as follows.

1) *State*: The state s_t at the decision moment t contains comprehensive information about the current scheduling environment, including jobs, factories, machines, and AGVs. In order to more accurately capture the complex correlations among different types of information, the disjunctive graph model is employed to represent the environment state, and the GNN is utilized to extract features. The state s_t of the environment is embedded into the following three types of nodes and two types of arc features.

The feature vector $\mu_{ij} \in \mathbb{R}^{10}$ of the operation node O_{ij} includes the following.

- 1) The current state value, i.e., scheduled (1) and unscheduled (0).
- 2) The number of available machines, i.e., $N_k(O_{ij}) = \sum_{F_f \in F_{ij}} |M_{ijf}|$.
- 3) The number of unscheduled operations for J_i .
- 4) The processing time of O_{ij} on machine M_{fk} , i.e., $P_{t_{ijfk}}$. If O_{ij} is scheduled, then $P_{t_{ijfk}} = P_{ijfk}$; otherwise, it is the estimated average processing time $P_{t_{ijfk}} = \sum_{F_f \in F_{ij}} \left(\sum_{M_{fk} \in M_{ijf}} P_{ijfk} / |M_{ijf}| \right)$. It is noted that $P_{t_{icfk}} = 0$ for the virtual operation node O_{ic} .
- 5) The start time for transportation $T_{s_{ij}} = \max\{EO_{ij}, EA_{fc}\}$, where EO_{ij} and EA_{fc} represent the earliest available time of O_{ij} and A_{fc} , respectively. If O_{ij} is the first operation of job i , then $EO_{ij} = 0$; if O_{ij} is not the first operation of job i and $O_{i(j-1)}$ is scheduled, then $EO_{ij} = Ct_{i(j-1)}$, where $Ct_{i(j-1)}$ represent the completion time of $O_{i(j-1)}$; if O_{ij} is not the first operation of job i and $O_{i(j-1)}$ is not scheduled, then $EO_{ij} = Est_{i(j-1)} + P_{t_{ijfk}}$, where $Est_{i(j-1)}$ is the estimated start time for processing of $O_{i(j-1)}$.
- 6) The end time for transportation $Te_{ij} = Ts_{ij} + T_c$, where T_c represent the transportation time of O_{ij} . If O_{ij} is scheduled, then $T_c = T_{ij}$; if $O_{i(j-1)}$ is scheduled and O_{ij} is not scheduled, then $T_c = ETc_{ij}$. The estimated average transportation time ETc_{ij} is calculated as follows:

$$ETc_{ij} = \frac{1}{N_k(O_{ij})} \sum_{F_f \in F_{ij}} \sum_{M_{fk} \in M_{ijf}} T_{ij}(M_{f'k'}, M_{fk}) \quad (1)$$

where $M_{f'k'}$ denotes the processing machine of $O_{i(j-1)}$, M_{fk} denotes the processing machine for O_{ij} , and $T_{ij}(M_{f'k'}, M_{fk})$ denotes the transportation time from $M_{f'k'}$ to M_{fk} . If neither $O_{i(j-1)}$ nor O_{ij} is scheduled, the estimated average transportation time T_c is calculated as follows:

$$ETc_{ij} = \frac{1}{N_k(O_{ij})} \sum_{F_{f'} \in F_{i(j-1)}} \sum_{M_{f'k'} \in M_{i(j-1)f}} \left[\frac{1}{N_k(O_{ij})} \sum_{F_f \in F_{ij}} \sum_{M_{fk} \in M_{ijf}} T_{ij}(M_{f'k'}, M_{fk}) \right]. \quad (2)$$

- 7) The start time for processing $St_{ij} = \max\{Te_{ij}, EM_{fk}\}$, where EM_{fk} represents the earliest available time of M_{fk} .
- 8) The completion time for processing $Ct_{ij} = St_{ij} + P_{t_{ijfk}}$.
- 9) The designated factory F_f .
- 10) The designated machine M_{fk} .

The feature vector $V_{fk} \in \mathbb{R}^6$ of the machine node M_{fk} includes: 1) number of processable operations for M_{fk} ; 2) earliest available time of M_{fk} ; 3) utilization rate of M_{fk} ; 4) workload of M_{fk} ; 5) factory F_f to which M_{fk} belongs; and 6) workload of F_f . For start and end nodes, their earliest available time and workload are set to 0, the utilization rate is set to 1, and the factory F_f is set to -1.

The feature vector $Y_{fc} \in \mathbb{R}^3$ of the AGV node A_{fc} includes: 1) factory F_f to which A_{fc} belongs (the F_f of CAGV is set to 0); 2) earliest available time of A_{fc} ; and 3) utilization rate of A_{fc} .

The feature of the O-M arc set represents the sum of the processing time $P_{t_{ijfk}}$ and the average transportation time T_c of the operation, and the feature of the O-A arc set represents the average transportation time T_c of the operation to the available machines.

2) *Action*: The action space $A(t)$ is composed of a set of mutually compatible action pairs, which represent all feasible O-M-A at decision point t . After each decision, O-M and O-A arc sets are updated to gradually reduce the action space. Furthermore, an AGV is still selected for the operation that does not require transportation to ensure the consistency of the actions selected at each scheduling step, but the AGV is not occupied.

3) *Reward*: The reward function is the feedback given by the environment after the action is executed, which directly guides the agent to choose a better scheduling strategy to gradually improve the quality of the solution. The objective of the DFJSPHT is to minimize the makespan, and the reward is defined as the difference between makespan before and after the state transition, i.e., $r(s_t, a_t, s_{t+1}) = C_{\max}(s_t) - C_{\max}(s_{t+1})$.

B. Resource Release Strategy

This article designs a release strategy based on resource states, aiming to rationally release the occupied resources to reduce the waiting time of AGVs and improve the collaboration efficiency of production-logistics. Specifically, if there are no feasible O-M-A in any batch at time t , the types of occupied resources are identified and can be categorized as machine

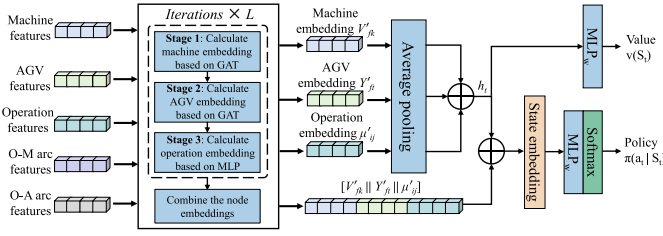


Fig. 5. Architecture of feature extraction.

occupancy, AGV occupancy, or both machine and AGV occupancy. When all available machines for the current jobs are occupied, the machine with the earliest completion time is immediately released at time t , and the status of the machines and the jobs being processed is updated synchronously. If there are still no feasible O-M-A after release, restore the occupied state of the machine and job, and advance time t to ensure the scheduling process continues. The release methods for the other two cases are similar to [23], and the detailed steps of the resource release strategy are given in “Section A of Supplementary Materials.”

C. Feature Embedding

In view of the two-stage embedding approach proposed in [23], this article designs an HGNN-based feature extraction architecture combining the characteristics of DFJSPHT to fully capture the complex relationships among nodes, as shown in Fig. 5. The process of node feature embedding is divided into three stages: 1) first, the operation nodes fuse the features of their adjacent AGV nodes, and then the machine nodes aggregate the embedding information of the corresponding operation nodes to form the semantically enhanced machine nodes representation; 2) subsequently, the operation nodes further integrate the embedding information of their associated machine nodes, and the AGV nodes aggregate their adjacent operation nodes to obtain their final representations; and 3) finally, the operation nodes integrate the embedded representations of adjacent operations, updated machine nodes and AGV nodes to realize the expression of final operation features through an MLP.

1) Machine Node Embedding:

The GAT is used to fuse the feature information of nodes and arcs in order to learn the correlation strength and importance relationship among different nodes. Specifically, the O-M arc is integrated into μ_{ij} to obtain the joint feature $\mu_{ijfk} = [\mu_{ij} \parallel S_{ijfk}]$, and the O-A arc is integrated into Y_{fc} to obtain the joint feature $\gamma_{ijfc} = [Y_{fc} \parallel \varphi_{ijfc}]$, where μ_{ij} represents the original feature of operation node O_{ij} , V_{fk} represents the original feature of machine node M_{fk} , Y_{fc} represents the original feature of AGV node A_{fc} , S_{ijfk} represents the original feature of O-M arc, and φ_{ijfc} represents the original feature of the O-A arc. The detailed calculation procedure for machine node embedding is as follows.

- 1) The attention coefficient of AGV node A_{fc} to operation node O_{ij} is calculated as follows:

$$e_{ijfc} = \text{LeakyReLU}(a^T \cdot [W^O \mu_{ij} \parallel W^P \gamma_{ijfc}]) \quad (3)$$

where $a^T \in \mathbb{R}^{2d}$, $W^O \in \mathbb{R}^{d \times 10}$, $\mu_{ij} \in \mathbb{R}^{10}$, $W^P \in \mathbb{R}^{d \times 4}$, and $\gamma_{ijfc} \in \mathbb{R}^4$.

- 2) The attention coefficient between operation nodes is calculated as follows:

$$e_{ij} = \text{LeakyReLU}(a^T \cdot [W^O \mu_{ij} \parallel W^O \mu_{ij}]) \quad (4)$$

- 3) The attention coefficient of operation node O_{ij} to machine node M_{fk} is calculated as follows:

$$e_{ijfk} = \text{LeakyReLU}(a^T \cdot [W^K V_{fk} \parallel W^L \mu_{ijfk}]) \quad (5)$$

where $W^K \in \mathbb{R}^{d \times 6}$, $V_{fk} \in \mathbb{R}^6$, $W^L \in \mathbb{R}^{d \times 11}$, and $\mu_{ijfk} \in \mathbb{R}^{11}$.

- 4) The attention coefficient between machine nodes is calculated as follows:

$$e_{fk} = \text{LeakyReLU}(a^T \cdot [W^K V_{fk} \parallel W^K V_{fk}]) \quad (6)$$

- 5) The above attention coefficients are normalized by the Softmax function to obtain α_{ijfc} , α_{ij} , α_{ijfk} , and α_{fk} . The operation node embedding that aggregates the features of AGV nodes is calculated as follows:

$$V_{ijfc} = \sigma \left(\alpha_{ij} W^O \mu_{ij} + \sum_{A_{fc} \in N_c(O_{ij})} \alpha_{ijfc} W^P \gamma_{ijfc} \right) \quad (7)$$

where $N_c(O_{ij})$ is the number of available AGVs for O_{ij} , and σ is the sigmoid function.

- 6) The final machine node embedding that aggregates the features of operation nodes is calculated as follows:

$$V'_{fk} = \sigma \left(\alpha_{fk} W^K V_{fk} + \sum_{O_{ij} \in N_o(M_{fk})} \alpha_{ijfk} (V_{ijfc} + W^E S_{ijfk}) \right) \quad (8)$$

where $N_o(M_{fk})$ is the number of processable operations for M_{fk} , and $W^E \in \mathbb{R}^{d \times 1}$.

2) AGV Node Embedding:

The GAT is still used to compute AGV node embeddings. The O-M arc is integrated into V'_{fk} to obtain the joint feature $h_{ijfk} = [V'_{fk} \parallel S_{ijfk}]$. By employing the same calculation method as machine node embedding, the attention coefficients obtained are as follows: x_{ijfk} from machine node M_{fk} to operation node O_{ij} , x_{ij} between operation nodes, x_{ijfc} from operation node O_{ij} to AGV node A_{fc} , and x_{cc} between AGV nodes.

- 1) The operation node embedding that aggregates the features of machine nodes is calculated as follows:

$$\mu'_{ijfk} = \sigma \left(x_{ij} W^O \mu_{ij} + \sum_{M_{fk} \in N_k(O_{ij})} x_{ijfk} W^H h_{ijfk} \right) \quad (9)$$

where $N_k(O_{ij})$ is the number of available machines for O_{ij} , $W^H \in \mathbb{R}^{d \times 7}$, and $h_{ijfk} \in \mathbb{R}^7$.

- 2) The final AGV node embedding that aggregates the features of operation nodes is calculated as follows:

$$Y'_{fc} = \sigma \left(x_{cc} W^T Y_{fc} + \sum_{O_{ij} \in N_o(A_{fc})} x_{ijfc} (\mu'_{ijfk} + W^E \varphi_{ijfc}) \right) \quad (10)$$

where $N_o(A_{fc})$ is the number of transportable operations for A_{fc} , $W^T \in \mathbb{R}^{d \times 3}$, and $Y_{fc} \in \mathbb{R}^3$.

3) Operation Node Embedding:

The multilayer perception (MLP₀, ..., MLP₅) and activation function ELU are utilized to directly process the feature information of O_{ij} and its neighbors, and finally an 8-D operation node embedding representation is obtained. The specific calculation formula is as follows:

$$\mu'_{ij} = \text{MLP}_0(\text{ELU}[\text{MLP}_1(O_{i,j-1}) \parallel \text{MLP}_2(O_{i,j}) \parallel \text{MLP}_3(O_{i,j+1}) \parallel \text{MLP}_4(\overline{V'_{fk}}) \parallel \text{MLP}_5(\overline{Y'_{fc}})]). \quad (11)$$

D. Action Prediction

The average pooling is performed on the final three types of node feature embeddings V'_{fk} , Y'_{fc} , and μ'_{ij} generated by the L-layer HGNN to obtain $h_t \in \mathbb{R}^{3d}$

$$h_t = \left[\frac{1}{|O|} \sum_{o_{ij} \in O} \mu'^{(L)}_{ij} \parallel \frac{1}{|M|} \sum_{M_{fk} \in M} V'^{(L)}_{fk} \parallel \frac{1}{|A|} \sum_{A_{fc} \in A} Y'^{(L)}_{fc} \right]. \quad (12)$$

Then, h_t is merged with the node feature embeddings as the input of MLP_w to generate the probability score P

$$P(a_t, s_t) = \text{MLP}_w[\mu'^{(L)}_{ij} \parallel V'^{(L)}_{fk} \parallel Y'^{(L)}_{fc} \parallel h_t] \quad (13)$$

where MLP_w consists of two d -dimensional hidden layers and an activation function tanh. Subsequently, the infeasible actions are masked by a mask mechanism, and the selection probability of each action in state s_t is obtained using the Softmax function

$$\pi(a_t | s_t) = \frac{\exp(P(a_t, s_t))}{\sum_{a'_t \in A(t)} \exp(P(a'_t, s_t))} \quad \forall a_t \in A(t). \quad (14)$$

E. Training Algorithm

To train the proposed DFJSPHT model, this article adopts the PPO with an actor-critic architecture proposed in [36], and its feasibility demonstration is given in "Section B of Supplementary Materials." In this framework, the actor network is responsible for generating action policies, while the critic network evaluates the value of state-action pairs to guide the iterative update of parameters, thereby facilitating the convergence toward increasingly optimized scheduling strategies. During training, the algorithm performs a total of 1000 iterations. In each episode, the agent interacts with the environment to collect experience. Simultaneously, the network parameters are updated every two iterations, and the current policy is evaluated every ten iterations using a validation set comprising 100 instances. For specific training procedure, refer to Algorithm 1.

V. EXPERIMENTS

A. Experiments Settings

1) Datasets:

In this article, two types of datasets (DS1 and DS2) are constructed for validating the performance of the proposed model. DS1 is used in the training and testing

Algorithm 1 Training Procedure

Input: HGNN architecture, actor network, critic network, training instance reset episodes N_r , policy update episodes N_u , policy validation episodes N_v , and trainable hyperparameters.

Output: The solution policy for DFJSPHT.

```

1: for  $iter = 1, 2, \dots, J$  do
2:   if  $iter \bmod N_r = 0$  then
3:     Generate the DFJSPHT instances with batch size  $B_n$ .
4:   end if
5:   for  $b = 1, 2, \dots, B_n$  do
6:     initialize state  $s_t$  based on instance  $b$ .
7:     while not terminal do
8:       Input the embedding extracted by HGNN into actor network.
9:       Sample  $a_t$  from policy  $\pi(a_t | s_t)$ .
10:      Record  $(s_t, a_t, r_t, s_{t+1})$  generated by interacting with the environment to memory.
11:      Advance  $t \rightarrow t + 1$ , and update  $s_t \rightarrow s_{t+1} + 1$ .
12:    end while
13:  end for
14:  if  $iter \bmod N_u = 0$  then
15:    Calculate PPO loss and update network parameters.
16:    Clear all memory.
17:  end if
18:  if  $iter \bmod N_v = 0$  then
19:    Validate the policy based on greedy strategy.
20:  end if
21: end for
22: return

```

phases, which includes 18 instances generated with a specific uniform distribution. DS1 is divided into three groups, each with the same job-machine combinations (i.e., 10×3 , 15×3 , 20×3 , 20×5 , 30×5 , and 40×5), but with different factory-IAGV-CAGV combinations (i.e., $2 \times 2 \times 2$, $2 \times 3 \times 2$, $3 \times 3 \times 3$, $3 \times 4 \times 3$, $4 \times 4 \times 4$, and $4 \times 5 \times 4$). In DS1, the instances with different numbers of machine correspond to the sets of operations and available machines obeying different uniform distributions, and different numbers of factories correspond to the sets of available factories obeying different uniform distributions. The processing time and transportation time in each instance obey the same uniform distribution, and the specific range of uniform distribution is shown in Table III. The training instances are dynamically generated during the training process (including 100 validation instances), while the test instances are sampled 100 instances for each scale. DS2 is constructed based on the classic FJSP benchmarks, namely the three types of la instances (rdata, edata, and vdata) in [37]. Each type of la instance (la01-la40) is expanded into three groups (G1-G3), with the number sets of factories and machines for G1-G3 are set as [2, 3, 4] and [3, 5, 7], respectively, and the number set of IAGVs and CAGVs is uniformly set as [2, 3, 4]. Furthermore, the uniform distribution corresponding

TABLE III
RANDOMLY GENERATED INSTANCE SCALE

Size	ops	fas	mas	pt	at	et
$10 \times 3 \times 2 \times 2 \times 2$	U(4,6)	U(1,2)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$15 \times 3 \times 2 \times 2 \times 2$	U(4,6)	U(1,2)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 3 \times 2 \times 2 \times 2$	U(4,6)	U(1,2)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 5 \times 2 \times 3 \times 2$	U(8,12)	U(1,2)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$30 \times 5 \times 2 \times 3 \times 2$	U(8,12)	U(1,2)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$40 \times 5 \times 2 \times 3 \times 2$	U(8,12)	U(1,2)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$10 \times 3 \times 3 \times 3 \times 3$	U(4,6)	U(1,3)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$15 \times 3 \times 3 \times 3 \times 3$	U(4,6)	U(1,3)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 3 \times 3 \times 3 \times 3$	U(4,6)	U(1,3)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 5 \times 3 \times 4 \times 3$	U(8,12)	U(1,3)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$30 \times 5 \times 3 \times 4 \times 3$	U(8,12)	U(1,3)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$40 \times 5 \times 3 \times 4 \times 3$	U(8,12)	U(1,3)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$10 \times 3 \times 4 \times 4 \times 4$	U(4,6)	U(1,4)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$15 \times 3 \times 4 \times 4 \times 4$	U(4,6)	U(1,4)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 3 \times 4 \times 4 \times 4$	U(4,6)	U(1,4)	U(1,3)	U(10,40)	U(10,20)	U(30,40)
$20 \times 5 \times 4 \times 5 \times 4$	U(8,12)	U(1,4)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$30 \times 5 \times 4 \times 5 \times 4$	U(8,12)	U(1,4)	U(1,5)	U(10,40)	U(10,20)	U(30,40)
$40 \times 5 \times 4 \times 5 \times 4$	U(8,12)	U(1,4)	U(1,5)	U(10,40)	U(10,20)	U(30,40)

Size($n \times m \times y \times r \times v$): number of jobs $\times$ number of machines in each factory $\times$ number of factories $\times$ number of IAGVs in each factory $\times$ number of CAGVs.

ops: number of operations for job.

fas: number of available factories for operation.

mas: number of available machines for operation in the available factories.

pt: processing time of operation.

at: transportation time among machines within the factory.

et: transportation time among factories.

to the transportation time among machines and among factories in D2 is the same as that in D1.

2) Configuration:

The same hyperparameters and model configurations are used for all instances of different scales. In the node feature embedding process, the HGNN network layer L is set to 2, and the embedding dimension is set to 8. The hidden layer dimension of the three-layer MLP used for operation node embedding is set to 128. In the model training process, the optimization period of the PPO is set to 3, and both its actor and critic networks are composed of three hidden layers with 64 dimensions. The loss weights of strategy, value, and entropy are set to 1, 0.5, and 0.01, respectively, and the discount factor is set to 1. Adam is chosen as the optimizer, and the learning rate is set to 2×10^{-4} . The hyperparameter sensitivity analysis is given in “Section C of Supplementary Materials”.

3) Comparison Methods:

This article selects two types of methods to compare with the proposed method to verify its effectiveness. The first type of methods consists of four classic priority dispatching rules (PDRs): first in first out (FIFO), shortest processing time (SPT), longest processing time (LPT), and most operations remaining (MOR). The second type of methods consists of two state-of-the-art DRL-based methods for solving FJSP, namely HGNN [23] and RL [25]. The main reasons for choosing these two

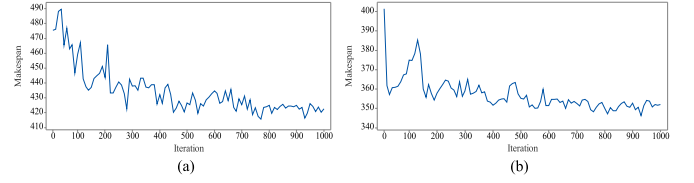


Fig. 6. Train curves on two instances. (a) $10 \times 3 \times 2 \times 2 \times 2$. (b) $10 \times 3 \times 3 \times 3 \times 3$.

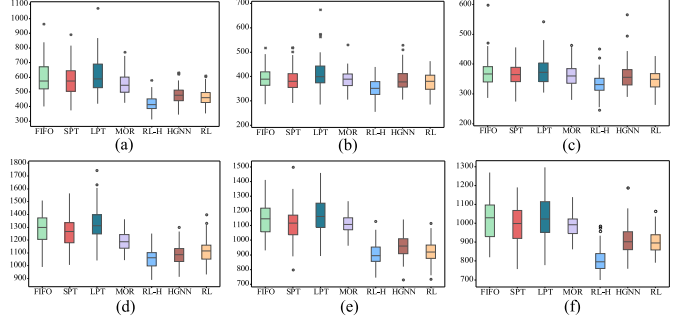


Fig. 7. Comparison results of all methods on six instances. (a) $10 \times 3 \times 2 \times 2 \times 2$. (b) $10 \times 3 \times 3 \times 3 \times 3$. (c) $10 \times 3 \times 4 \times 4 \times 4$. (d) $20 \times 5 \times 2 \times 3 \times 2$. (e) $20 \times 5 \times 3 \times 4 \times 3$. (f) $20 \times 5 \times 4 \times 5 \times 4$.

comparison methods are as follows: 1) all four PDR methods are widely used to evaluate the performance of the newly proposed DRL-based algorithm for solving the scheduling problems; 2) HGNN is a classic DRL method combining GNN to solve FJSP; and 3) RL is a recently proposed DRL method combining GNN for solving the FJSP with transportation constraints. Given that the problem studied in this article is DFJSPHT, the factory-level information and the heterogeneous AGVs feature information are integrated into the network structures of HGNN and RL to ensure the fairness of the comparison experiments. For the convenience of presenting the experimental results, the method proposed in this article is denoted as RL-H. In addition, the environment for the four PDRs methods is set to be the same as RL-H.

In terms of the performance evaluation metric for comparison methods, the gap metric is used to measure the gap between the makespan ($C_{\max}$) of the current method and the best makespan (C_b) among all methods. The gap value is calculated by the following equation:

$$\text{Gap} = \frac{C_{\max} - C_b}{C_b} \times 100\%. \quad (15)$$

B. Experiment Results on DS1

Based on the synthesized instance DS1, RL-H, HGNN, and RL were trained on six different scale instances ($10 \times 3 \times 2 \times 2 \times 2$, $10 \times 3 \times 3 \times 3 \times 3$, $10 \times 3 \times 4 \times 4 \times 4$, $20 \times 5 \times 2 \times 3 \times 2$, $20 \times 5 \times 3 \times 4 \times 3$, and $20 \times 5 \times 4 \times 5 \times 4$), and the obtained models were directly used to solve the instances with the same number of factories and machines. For example, a model trained on $10 \times 3 \times 2 \times 2 \times 2$ is used to solve the $10 \times 3 \times 2 \times 2 \times 2$, $15 \times 3 \times 2 \times 2 \times 2$, and $20 \times 3 \times 2 \times 2 \times 2$. In terms of training time, RL-H takes 0.63, 1.24, 1.34, 4.52, 5.12, and 5.56 h on $10 \times 3 \times 2 \times 2 \times 2$, $10 \times 3 \times 3 \times 3 \times 3$,

TABLE IV
COMPARISON RESULTS ON SYNTHETIC INSTANCES

Size		FIFO	SPT	LPT	MOR	RL-H	HGNN	RL
$10 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	593.97	580.54	609.91	555.02	419.59	479.58	464.78
	Gap	41.56%	38.36%	45.36%	32.27%	0.00%	14.30%	10.77%
$15 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	896.05	951.87	890.87	845.41	549.09	643.40	638.25
	Gap	63.19%	73.35%	62.25%	53.97%	0.00%	17.18%	16.24%
$20 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	1145.60	1208.27	1114.20	1027.64	690.24	826.01	815.95
	Gap	65.97%	75.05%	61.42%	48.88%	0.00%	19.67%	18.21%
$20 \times 5 \times 2 \times 3 \times 2$	$C_{\max}$	1284.23	1270.81	1324.83	1187.32	1055.64	1090.75	1112.62
	Gap	21.65%	20.38%	25.50%	12.47%	0.00%	3.33%	5.40%
$30 \times 5 \times 2 \times 3 \times 2$	$C_{\max}$	1784.75	1779.77	1852.69	1642.31	1460.08	1512.45	1521.50
	Gap	22.24%	21.90%	26.89%	12.48%	0.00%	3.59%	4.21%
$40 \times 5 \times 2 \times 3 \times 2$	$C_{\max}$	2256.98	2286.92	2377.94	2230.34	1968.06	2070.27	2138.93
	Gap	14.68%	16.20%	20.83%	13.33%	0.00%	5.19%	8.68%
$10 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	392.21	389.63	408.79	389.90	351.09	386.16	376.83
	Gap	11.71%	10.98%	16.43%	11.05%	0.00%	9.99%	7.33%
$15 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	588.16	562.05	610.04	521.17	448.09	479.34	491.99
	Gap	31.26%	25.43%	36.14%	16.31%	0.00%	6.97%	9.80%
$20 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	848.47	814.42	876.60	719.54	554.96	695.59	654.83
	Gap	52.89%	46.75%	57.96%	29.65%	0.00%	25.34%	18.00%
$20 \times 5 \times 3 \times 4 \times 3$	$C_{\max}$	1144.84	1114.66	1166.11	1110.03	902.92	957.32	917.69
	Gap	26.79%	23.45%	29.15%	22.94%	0.00%	6.02%	1.64%
$30 \times 5 \times 3 \times 4 \times 3$	$C_{\max}$	1571.23	1534.38	1575.07	1468.52	1307.28	1378.51	1362.74
	Gap	20.19%	17.37%	20.48%	12.33%	0.00%	5.45%	4.24%
$40 \times 5 \times 3 \times 4 \times 3$	$C_{\max}$	1947.25	1950.84	1986.13	1920.41	1725.32	1819.54	1816.87
	Gap	12.86%	13.07%	15.12%	11.31%	0.00%	5.46%	5.31%
$10 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	368.63	365.83	377.09	361.01	331.34	359.11	344.55
	Gap	11.26%	10.41%	13.81%	8.96%	0.00%	8.38%	3.99%
$15 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	452.22	460.02	462.19	445.84	387.83	428.99	414.84
	Gap	16.60%	18.61%	19.17%	14.96%	0.00%	10.61%	6.96%
$20 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	570.61	545.93	591.33	539.58	449.90	512.56	489.59
	Gap	26.83%	21.34%	31.44%	19.93%	0.00%	13.93%	8.82%
$20 \times 5 \times 4 \times 5 \times 4$	$C_{\max}$	1019.72	992.77	1027.72	986.22	804.07	911.14	899.67
	Gap	26.82%	23.47%	27.81%	22.65%	0.00%	13.32%	11.89%
$30 \times 5 \times 4 \times 5 \times 4$	$C_{\max}$	1360.02	1324.60	1388.33	1293.32	1097.24	1235.97	1218.17
	Gap	23.95%	20.72%	26.53%	17.87%	0.00%	12.64%	11.02%
$40 \times 5 \times 4 \times 5 \times 4$	$C_{\max}$	1706.03	1686.78	1723.90	1660.86	1380.37	1497.21	1440.13
	Gap	23.59%	22.20%	24.89%	20.32%	0.00%	8.46%	4.33%

$10 \times 3 \times 4 \times 4 \times 4$, $20 \times 5 \times 2 \times 3 \times 2$, $20 \times 5 \times 3 \times 4 \times 3$ and $20 \times 5 \times 4 \times 5 \times 4$, respectively. In addition, the training curves of RL-H on $10 \times 3 \times 2 \times 2 \times 2$, and $10 \times 3 \times 3 \times 3 \times 3$ are provided. From Fig. 6, it can be observed that there is some fluctuation in the early stages of training, but the overall trend is relatively stable and converges quickly.

The experimental results of RL-H and other comparison methods are depicted in Table IV, which demonstrate that the proposed RL-H has a significant advantage in all instances. It is worth noting that the relative gap between RL-H and other comparison methods gradually narrows as the number of factories increases when solving the instances with three machines ($m = 3$). The fundamental reason is that the proposed resource release strategy can effectively reduce the waiting time of AGVs caused by limited machine resources to shorten the makespan, and its effectiveness is verified in Section V-E. However, as the number of factories increases, the available machines for each operation expand synchronously, alleviating the waiting phenomenon of AGVs. Therefore, the relative gap between RL-H and other comparison methods gradually

decreases. Furthermore, the makespan distributions obtained by all methods on six training instances of different scales are shown in Fig. 7. It can be observed that RL-H has significantly better solution results than other comparison methods on six instances, and exhibits good adaptability and stability. Due to space limitations, the average running time and results analysis of the RL-H and the comparison methods are provided in “Section D of Supplementary Materials.”

C. Generalization Evaluation on Large-Scale Instances

To assess the cross-scale generalization capability of RL-H, nine groups of large-scale test instances were constructed, each consisting of 100 randomly generated instances with the same range of values as the instances with five machines in Table III. Three models trained on the instances $10 \times 5 \times 5 \times 5 \times 5$, $10 \times 5 \times 6 \times 6 \times 6$, and $10 \times 5 \times 7 \times 7 \times 7$ were directly deployed to solve all 900 large-scale instances. It is worth mentioning that each model is only used to solve the instances with the same number of factories as it. Furthermore, the comparison methods HGNN and RL were trained and

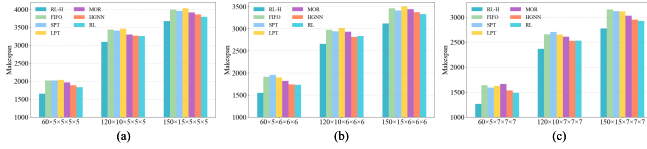


Fig. 8. Average makespan on large-scale instances. (a) Five-factory instances. (b) Six-factory instances. (c) Seven-factory instances.

TABLE V
COMPARISON RESULTS ON LA INSTANCES

Instances			MOR	RL-H	HGNN	RL
la(edata)	G1	$C_{\max}$	2054.66	1824.56	1944.85	1963.81
		Gap	12.61%	0.00%	6.59%	7.63%
	G2	$C_{\max}$	1417.55	1082.36	1247.86	1192.52
		Gap	30.97%	0.00%	15.29%	10.18%
	G3	$C_{\max}$	1035.94	887.71	993.09	971.61
		Gap	16.70%	0.00%	11.87%	9.45%
la(rdata)	G1	$C_{\max}$	2074.38	1856.83	1982.59	1955.73
		Gap	11.72%	0.00%	6.77%	5.33%
	G2	$C_{\max}$	1434.29	1060.64	1254.54	1199.23
		Gap	35.23%	0.00%	18.28%	13.07%
	G3	$C_{\max}$	1012.49	873.49	989.52	952.19
		Gap	15.91%	0.00%	13.28%	9.01%
la(vdata)	G1	$C_{\max}$	2097.06	1831.91	1978.56	1951.99
		Gap	14.47%	0.00%	8.01%	6.55%
	G2	$C_{\max}$	1439.50	1060.15	1229.13	1183.60
		Gap	35.78%	0.00%	15.94%	11.64%
	G3	$C_{\max}$	1094.61	868.89	990.12	966.75
		Gap	25.98%	0.00%	13.95%	11.26%

tested on the same instances to ensure the fairness of the experiments.

Fig. 8 presents the average makespan results of all methods on instances with different numbers of factories. As illustrated, RL-H consistently achieves the minimum makespan on all large-scale instances, confirming its superior generalization capability compared to other comparative methods in large-scale problem scenarios. Due to space limitations, specific gap values and runtime are provided in “Section E of Supplementary Materials.”

D. Experiment Results on DS2

In order to further validate the performance of the proposed method on unknown instances, DS2 was constructed based on the public FJSP benchmark for testing. The models trained in Section V-B were directly applied to the three groups of instances (G1–G3) of la instances without further training or fine-tuning. It is worth noting that DS2 is completely different from the training dataset, which can evaluate the generalization ability of algorithm in the production scenarios of other scales.

The experimental results of all methods on the la instance are shown in Table V. It can be seen that the solutions obtained by RL-H significantly outperform other comparison methods in all instances of DS2. In most instances, the RL-H reduces the makespan by more than 15% compared to the MOR rule and by more than 5% compared to the comparison DRL methods. Furthermore, the average makespan generated by RL-H on three types of la instances is relatively stable. As shown

TABLE VI
EVALUATION RESULTS OF RESOURCE RELEASE STRATEGY

Size		RL-H	RL-H1	HGNN	RL
$40 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	1258.35	1480.78	1531.62	1507.34
	Gap	0.00%	17.68%	21.72%	19.79%
$80 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	2358.54	2803.14	2920.57	2883.90
	Gap	0.00%	18.85%	23.83%	22.27%
$100 \times 3 \times 2 \times 2 \times 2$	$C_{\max}$	2891.60	3482.43	3636.16	3582.79
	Gap	0.00%	20.43%	25.75%	23.90%
$40 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	1019.53	1079.65	1131.77	1148.61
	Gap	0.00%	5.90%	11.01%	12.66%
$80 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	1949.29	2103.83	2093.04	2163.02
	Gap	0.00%	7.93%	7.37%	10.96%
$100 \times 3 \times 3 \times 3 \times 3$	$C_{\max}$	2491.46	2590.29	2681.99	2677.48
	Gap	0.00%	3.97%	7.65%	7.47%
$40 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	713.27	867.92	870.85	880.74
	Gap	0.00%	21.68%	22.09%	23.48%
$80 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	1346.34	1512.90	1600.04	1625.10
	Gap	0.00%	12.37%	18.84%	20.71%
$100 \times 3 \times 4 \times 4 \times 4$	$C_{\max}$	1693.36	1881.43	1963.05	1979.81
	Gap	0.00%	11.11%	15.93%	16.92%

in Table S-VI of “Section F of Supplementary Materials,” although the RL-H is slightly higher than other comparison methods in terms of average running time, its impact on the scheduling efficiency of real distributed manufacturing systems is acceptable. This experiment proves that the RL-H can ensure the quality of the scheduling schemes while also having the adaptability to different problem scenarios.

E. Effectiveness Evaluation of Resource Release Strategy

To evaluate the contribution of the proposed resource release strategy to algorithm performance, the RL-H with the release strategy is compared with the RL-H without the release strategy (denoted as RL-H1), HGNN, and RL. For the experimental design, the models trained on instances $10 \times 3 \times 2 \times 2 \times 2$, $10 \times 3 \times 3 \times 3 \times 3$, and $10 \times 3 \times 4 \times 4 \times 4$ are directly applied to the instances with the same number of factories and machines to evaluate the effectiveness of the release strategy in scenarios where machine resources are relatively scarce ($m = 3$). In addition, RL-H1 is also trained on the same instances to obtain corresponding models. The experimental results for all comparison methods are presented in Table VI.

As shown in Table VI, RL-H achieved the best results in all instances. Second, RL-H1 demonstrates superior performance to both HGNN and RL in most instances, ranking second only to RL-H. A paired t-test was performed to verify the significance of RL-H results, and the statistical results are recorded in “Section G of Supplementary Materials.” The mean paired differences highlight that RL-H can always achieve smaller makespans, and all p values are far below the significance threshold (0.05), which fully proves that RL-H has significant advantages in solving performance compared to the other three comparison methods. In addition, in order to represent the scheduling results of instances more clearly, Gantt charts were generated based on instances $40 \times 3 \times 2 \times 2 \times 2$ by RL-H and RL-H1 as shown in “Section H of Supplementary Materials.”

From the above experiment, the following conclusions can be drawn: 1) in a distributed manufacturing scenario with a certain number of AGVs, the proposed resource release strategy can effectively enhance the efficiency of AGVs to reduce the makespan; 2) the proposed network architecture is more efficient than other comparison methods in solving DFJSPHT; and 3) RL-H has excellent generalization ability in scenarios of different scales.

VI. CONCLUSION

This article proposes an end-to-end DRL intelligent scheduling method to solve the proposed DFJSPHT. First, a novel heterogeneous disjunction graph model is constructed to uniformly represent the state information and correlation relationships of factories, machines, operations, and transportation resources in DFJSPHT. Then, a three-stage feature embedding method integrating GAT and MLP is proposed to efficiently extract the feature embedding information of three types of nodes. In addition, a resource release strategy is designed to enhance the collaboration efficiency between transportation and production. Finally, the extracted feature embedding is used as the input of the policy-value network, and the PPO is employed for training to obtain high-quality scheduling policies. The proposed method was tested on randomly generated instances and public benchmark instances, and the experimental results validate its effectiveness and generalization performance in solving DFJSPHT.

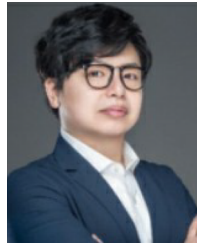
Overall, the research value of this study is mainly reflected in two aspects. On the one hand, the proposed DFJSPHT extends the scope of traditional DFJSP research, providing a reference framework for state modeling and policy learning in subsequent multiconstraint distributed scheduling scenarios. On the other hand, the research findings can provide efficient and feasible scheduling solutions for multifactory production-logistics collaboration in real manufacturing systems, which has significant theoretical significance and application value for promoting the automation and intelligence development of manufacturing processes.

In future research, we will focus on the following directions: 1) considering dynamic factors in real-world production, such as urgent orders, machine failures, and quality anomalies; 2) exploring new methods to enhance model generalization and solution performance based on multiagent technology; and 3) incorporating the differences in load weight, speed, and energy consumption of transportation resources.

REFERENCES

- [1] C. Wang, M. Wei, Q. Liu, X. Zhang, and X. Li, "An improved adaptive hybrid algorithm for solving distributed flexible job shop scheduling problem," *Swarm Evol. Comput.*, vol. 94, Apr. 2025, Art. no. 101873.
- [2] X. Li et al., "A Q-learning improved differential evolution algorithm for human-centric dynamic distributed flexible job shop scheduling problem," *J. Manuf. Syst.*, vol. 80, pp. 794–823, Jun. 2025.
- [3] K. Zhu et al., "Dynamic distributed flexible job-shop scheduling problem considering operation inspection," *Expert Syst. Appl.*, vol. 224, Aug. 2023, Art. no. 119840.
- [4] H. Mu, M. Cheng, X. Tang, R. Ding, and W. Ma, "A hybrid distributed-centralized load sensing system for efficiency improvement of electrified construction machinery," *Energy*, vol. 314, Jan. 2025, Art. no. 134123.
- [5] M. Yuan, S. Lu, L. Zheng, Q. Yu, F. Pei, and W. Gu, "Distributed heterogeneous flexible job-shop scheduling problem considering automated guided vehicle transportation via improved deep q network," *Swarm Evol. Comput.*, vol. 94, Apr. 2025, Art. no. 101902.
- [6] X. Zhou, F. Wang, B. Wu, Y. Li, and N. Shen, "Deep reinforcement learning-based memetic algorithm for solving dynamic distributed green flexible job shop scheduling problem with finite transportation resources," *Swarm Evol. Comput.*, vol. 94, Apr. 2025, Art. no. 101885.
- [7] Z. Pan, L. Wang, J. Wang, and Q. Zhang, "A bi-learning evolutionary algorithm for transportation-constrained and distributed energy-efficient flexible scheduling," *IEEE Trans. Evol. Comput.*, vol. 29, no. 1, pp. 232–246, Feb. 2025.
- [8] N. Peng et al., "Multi-objective dynamic distributed flexible job shop scheduling problem considering uncertain processing time," *Cluster Comput.*, vol. 28, no. 3, p. 185, Jun. 2025.
- [9] M. Samhoury and S. Z. Qareish, "Hybrid fuzzy genetic algorithm for the integration of process planning and scheduling for distributed flexible job shop," *Neural Comput. Appl.*, vol. 37, no. 4, pp. 2775–2798, Feb. 2025.
- [10] X. Li, S. Zhang, P. Jiang, M. Deng, X. V. Wang, and C. Yin, "Knowledge graph based OPC UA information model automatic construction method for heterogeneous devices integration," *Robot. Comput.-Integr. Manuf.*, vol. 88, Aug. 2024, Art. no. 102736.
- [11] Y. Sang and J. Tan, "Intelligent factory many-objective distributed flexible job shop collaborative scheduling method," *Comput. Ind. Eng.*, vol. 164, Feb. 2022, Art. no. 107884.
- [12] J. Xie, X. Li, L. Gao, and L. Gui, "A hybrid genetic Tabu search algorithm for distributed flexible job shop scheduling problems," *J. Manuf. Syst.*, vol. 71, pp. 82–94, Dec. 2023.
- [13] F. Yu, C. Lu, J. Zhou, L. Yin, and K. Wang, "A knowledge-guided bi-population evolutionary algorithm for energy-efficient scheduling of distributed flexible job shop problem," *Eng. Appl. Artif. Intell.*, vol. 128, Feb. 2024, Art. no. 107458.
- [14] Y. Fu, K. Gao, L. Wang, M. Huang, Y.-C. Liang, and H. Dong, "Scheduling stochastic distributed flexible job shops using an multi-objective evolutionary algorithm with simulation evaluation," *Int. J. Prod. Res.*, vol. 63, no. 1, pp. 86–103, Jan. 2025.
- [15] B. Marzouki, O. B. Driss, and K. Ghedira, "Improved chemical reaction optimization for distributed flexible job shop problem with transportation times," *IFAC-PapersOnLine*, vol. 55, no. 10, pp. 1249–1254, 2022.
- [16] Q. Luo, Q. Deng, G. Gong, L. Zhang, W. Han, and K. Li, "An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers," *Expert Syst. Appl.*, vol. 160, Dec. 2020, Art. no. 113721.
- [17] J. Yang and H. Xu, "Hybrid memetic algorithm to solve multiobjective distributed fuzzy flexible job shop scheduling problem with transfer," *Processes*, vol. 10, no. 8, p. 1517, Aug. 2022.
- [18] J. Ma, W. Gao, and W. Tong, "A deep reinforcement learning assisted adaptive genetic algorithm for flexible job shop scheduling," *Eng. Appl. Artif. Intell.*, vol. 149, Jun. 2025, Art. no. 110447.
- [19] J.-P. Huang, L. Gao, and X.-Y. Li, "A hierarchical multi-action deep reinforcement learning method for dynamic distributed job-shop scheduling problem with job arrivals," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 2501–2513, 2025.
- [20] J.-P. Huang, L. Gao, and X.-Y. Li, "An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem," *Expert Syst. Appl.*, vol. 238, Mar. 2024, Art. no. 121756.
- [21] E. Yuan, L. Wang, S. Cheng, S. Song, W. Fan, and Y. Li, "Solving flexible job shop scheduling problems via deep reinforcement learning," *Expert Syst. Appl.*, vol. 245, Jul. 2024, Art. no. 123019.
- [22] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 3020–3038, Oct. 2022.
- [23] W. Song, X. Chen, Q. Li, and Z. Cao, "Flexible job-shop scheduling via graph neural network and deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1600–1610, Feb. 2023.
- [24] L. Wan, L. Fu, C. Li, and K. Li, "Flexible job shop scheduling via deep reinforcement learning with meta-path-based heterogeneous graph neural network," *Knowl.-Based Syst.*, vol. 296, Jul. 2024, Art. no. 111940.
- [25] Y. Wang, R. Wang, J. Sun, F. Deng, G. Wang, and J. Chen, "Attention enhanced reinforcement learning for flexible job shop scheduling with transportation constraints," *Expert Syst. Appl.*, vol. 282, Jul. 2025, Art. no. 127671.
- [26] Y. Du, J. Li, C. Li, and P. Duan, "A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5695–5709, Apr. 2024.
- [27] W. Wang, Y. Zhang, Y. Wang, G. Pan, and Y. Feng, "Hierarchical multi-agent deep reinforcement learning for dynamic flexible job-shop scheduling with transportation," *Int. J. Prod. Res.*, vol. 63, pp. 1–28, Jun. 2025.

- [28] M. Yuan, L. Zheng, H. Huang, K. Zhou, F. Pei, and W. Gu, "Research on flexible job shop scheduling problem with AGV using double DQN," *J. Intell. Manuf.*, vol. 36, no. 1, pp. 509–535, Jan. 2025.
- [29] J. Shi, F. Qiao, J. Liu, Y. Ma, D. Wang, and C. Ding, "Production-logistics collaborative scheduling in dynamic flexible job shops using nested-hierarchical deep reinforcement learning," *Adv. Eng. Informat.*, vol. 65, May 2025, Art. no. 103195.
- [30] Y. Li et al., "Real-time scheduling for flexible job shop with AGVs using multiagent reinforcement learning and efficient action decoding," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 3, pp. 2120–2132, Mar. 2025.
- [31] R. Li, W. Gong, L. Wang, C. Lu, and C. Dong, "Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 1, pp. 201–211, Jan. 2024.
- [32] Y. Yan, W. Yi, Z. Pei, and Y. Chen, "Multi-agent reinforcement learning for distributed flexible job shop scheduling with random job arrival," *IEEE Access*, vol. 13, pp. 80941–80957, 2025.
- [33] J. Li, J. Li, and Y. Xu, "HGNN: A PCA-based heterogeneous graph neural network for a family distributed flexible job shop," *Comput. Ind. Eng.*, vol. 200, Feb. 2025, Art. no. 110855.
- [34] L. Li and Z. Chai, "Energy-saving distributed flexible job-shop scheduling with fuzzy processing time in IIoT: A novel evolutionary multitasking algorithm," *J. Ind. Inf. Integr.*, vol. 45, May 2025, Art. no. 100829.
- [35] Z.-Q. Zhang, F.-C. Wu, B. Qian, R. Hu, L. Wang, and H.-P. Jin, "A Q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation," *Expert Syst. Appl.*, vol. 234, Dec. 2023, Art. no. 121050.
- [36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [37] J. Hurink, B. Jurisch, and M. Thole, "Tabu search for the job-shop scheduling problem with multi-purpose machines," *Oper.-Res.-Spektrum*, vol. 15, no. 4, pp. 205–215, Dec. 1994.



Pei Jiang (Member, IEEE) received the B.S. and Ph.D. degrees in control science and engineering from Zhejiang University, Hangzhou, China, in 2008 and 2015, respectively.

He is currently a Professor with the College of Mechanical and Vehicle Engineering, Chongqing University, Chongqing, China. His research interests include robotic systems, soft robotics, and energy modeling of industrial robots.



Min Cheng was born in Jingzhou, China, in 1987. He received the B.S. and Ph.D. degrees in mechatronics from Zhejiang University, Hangzhou, China, in 2009 and 2015, respectively.

He is currently a Professor with the State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, Chongqing, China. His research interests include electrohydraulic control systems of mobile machinery, energy saving and motion control of hydraulic systems, and mechatronic systems design.



Yuanqing Wu (Senior Member, IEEE) received the Ph.D. degree from the Department of Control Science and Engineering, Zhejiang University, Hangzhou, China, in 2016.

He was a Research Assistant with the School of Electrical and Electronic Engineering, Nanyang Technology University, Singapore, from 2014 to 2015. He is currently a Full Professor with the School of Intelligent Engineering, Sun Yat-sen University, Guangzhou, China. His research interests include nonlinear control systems, networked control

systems, output regulation, and artificial intelligence.



Kaikai Zhu received the B.S. degree in industrial engineering from Dalian Polytechnic University, Dalian, China, in 2019, and the M.S. degree in mechanical engineering from the Central South University of Forestry and Technology, Changsha, China, in 2024. He is currently pursuing the Ph.D. degree with the Chongqing University, Chongqing, China.

His research interests include scheduling optimization and reinforcement learning.



Kaizhou Gao (Member, IEEE) received the Ph.D. degree from Nanyang Technological University (NTU), Singapore, in 2016.

He is currently an Assistant Professor with Macau Institute of Systems Engineering, Macau University of Science and Technology, Macau, China. His research interests include intelligent optimization and scheduling.



Xiaobin Li (Member, IEEE) received the B.S. degree in geophysics from China University of Mining and Technology, Xuzhou, China, in 2010, and the Ph.D. degree in mechanical engineering from Chongqing University, Chongqing, China, in 2015.

He is currently an Associate Professor with the College of Mechanical and Vehicle Engineering, Chongqing University. His research interests include manufacturing system engineering, intelligent manufacturing, and robotic systems.



Lei Ren (Member, IEEE) received the Ph.D. degree in computer science from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2009.

He is currently a Professor with the School of Automation Science and Electrical Engineering, Beihang University, Beijing. He is also with the State Key Laboratory of Intelligent Manufacturing Systems Technology, Beijing, and Zhongguancun Laboratory, Beijing. His research interests include neural networks and deep learning, industrial digital

twin, and industrial AI applications.

Dr. Ren serves as an Associate Editor for IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS and other international journals.